



**Universidad Autónoma de
Nayarit**

Unidad Académica De Economía

Sistemas Computacionales

**“Semana 10 — Quick Sort y Veredicto
StreamMX”**

Jorge Armando Navarrete Ruiz

Docente.- DR. Eligardo Cruz Sanchez

U. A. ESTRUCTURA DE BASES DE DATOS

Fase 1: COMPRENDE	3
Checkpoint COMPRENDE — 4 preguntas metacognitivas	3
Reto 1A — Comprende: Cuestionamiento socrático sobre Quick Sort	4
Fase 2: APLICA	11
Reto 3 — Quick Sort + Benchmark comparativo	11
Reto 4 — Quick Sort configurable: tres estrategias de pivote	11
Fase 3: REFLEXIONA	11
Reto 1A — Reflexiona: Simulacro de Defensa Oral (Merge Sort)	11
Fase 4: VALIDA	11
Reto 1A — Válida: Diseño de Casos Adversariales para Merge Sort	11

Fase 1: COMPRENDE

Checkpoint COMPRENDE — 4 preguntas metacognitivas

¿Cuál es la diferencia fundamental entre cómo Merge Sort y Quick Sort dividen el problema? (Pista: ¿quién "hace el trabajo" — la división o la combinación?)

Merge Sort divide el arreglo en mitades por posición (independiente de los valores). La división es "barata" y el trabajo fuerte está en la combinación: toma dos mitades ordenadas y las fusiona en $O(n)$

Quick Sort divide el arreglo por valores alrededor de un pivote. El trabajo fuerte está en la división: reordena el arreglo para que el pivote quede en su lugar final y separa según la variante).

En la traza de la partición Lomuto, ¿por qué el pivote nunca vuelve a moverse después de colocarse en la posición $i+1$? ¿Qué garantiza el invariante?

Al terminar el bucle (), se hace el swap final: $j == high$

`swap(A[i+1], A[high])`

Por qué no se mueve después: porque esa operación coloca al pivote en el índice tal que: $p = i+1$

todo a la izquierda es $A[low..p-1] < pivot$

todo a la derecha es $A[p+1..high] \geq pivot$

Eso significa que el pivote ya está en su posición final dentro de . Las llamadas recursivas operan en y , que excluyen el pivote, así que no vuelve a tocarse.

Si el arreglo de entrada es [5, 4, 3, 2, 1] y usamos Lomuto con pivote = último elemento, ¿cuántas comparaciones se realizan? ¿Por qué esto es $O(n^2)$?

Por qué es $O(n^2)$: en el peor caso, el pivote termina siempre en un extremo y la recursión queda desbalanceada

¿Por qué un arreglo de puros duplicados [7, 7, 7, 7, 7] también degenera en $O(n^2)$ con Lomuto? Traza los primeros dos pasos de la partición para verificar.

Con muchos duplicados y partición de 2 vías, los iguales al pivote se van todos al mismo lado, generando particiones extremadamente desbalanceadas.

Reto IA — Comprende: Cuestionamiento socrático sobre Quick Sort

```
#!/usr/bin/env python3
```

```
"""
```

```
Semana 9 — Divide y Vencerás: Merge Sort
```

```
Curso: Estructura de Datos Básica
```

```
Equipo: [Nombre del equipo]
```

```
Integrantes: Jorge Armando Navarrete Ruiz
```

```
Fecha: [Fecha de entrega]
```

```
"""
```

```
import time
```

```
import random
```

```
import copy
```

```
#
```

```
_____
```

```
_____
```

```
# SECCIÓN 1: MERGE SORT
```

```
#
```

```
_____
```

```
_____
```

```
# COMPRENDE-1: ¿Qué propiedad garantiza que merge() siempre
```

```
#     produce un subarreglo ordenado?
```

```
# Porque las dos mitades que merge() fusiona ya están ordenadas, y en cada paso
```

```
# se elige el menor de los dos elementos “al frente” (izquierda[i] vs derecha[j]).
```

```
# Al tomar siempre el mínimo disponible, el subarreglo resultante se construye en orden.
```

```
# COMPRENDE-2: ¿Por qué el árbol de recursión tiene altura  $\log_2(n)$ 
```

y qué implica eso para la complejidad total?
 # Porque en cada llamada el problema se divide aproximadamente a la mitad, así que
 # la cantidad de niveles hasta llegar a subarreglos de tamaño 1 es $\log_2(n)$.
 # Como en cada nivel se fusionan en total n elementos, la complejidad total es $O(n \log n)$.

```
def merge_sort(arr: list, inicio: int, fin: int) -> None:
```

```
    """
```

```
    Ordena arr[inicio..fin] usando Merge Sort.
```

DECISIÓN: Usamos índices inicio/fin para evitar crear muchos subarreglos con slicing

en cada llamada (más tiempo y memoria). Así ordenamos sobre el mismo arreglo

y solo usamos memoria auxiliar dentro de merge().

```
    """
```

```
    if inicio >= fin:
```

```
        return # Caso base: 0 o 1 elemento ya está ordenado
```

```
    # TODO: calcular el punto medio (división entera)
```

```
    medio = ...
```

```
    merge_sort(arr, ..., ...) # TODO: llamada recursiva mitad izquierda
```

```
    merge_sort(arr, ..., ...) # TODO: llamada recursiva mitad derecha
```

```
    merge(arr, inicio, medio, fin)
```

```
def merge(arr: list, inicio: int, medio: int, fin: int) -> None:
```

```
    """
```

```
    Fusiona dos subarreglos ordenados: arr[inicio..medio] y arr[medio+1..fin].
```

DECISIÓN: Copiamos en arreglos auxiliares porque durante la fusión vamos sobrescribiendo arr;

si intentáramos leer y escribir “encima” sin copia, podríamos pisar valores que aún

necesitamos comparar. Los auxiliares garantizan que los datos de entrada se mantienen intactos.

```
    """
```

```
    # COMPRENDE-3: Invariante del ciclo de fusión:
```

```
    # Al finalizar cada iteración, arr[inicio..k-1] contiene los k-inicio
```

```
    # elementos más pequeños de izquierda  $\cup$  derecha, en orden.
```

Confirmación: Sí. Tras cada iteración, arr[inicio..k-1] queda ordenado y contiene exactamente

los (k-inicio) elementos más pequeños de (izquierda $\cup$ derecha). Lo pendiente por

decidir

permanece en izquierda[i..] y derecha[j..].

izquierda = arr[inicio:medio + 1]

derecha = arr[medio + 1:fin + 1]

i = 0; j = 0; k = inicio

while i < len(izquierda) and j < len(derecha):

DECISIÓN: Usamos <= (y no <) para mantener estabilidad: si hay empate,

toma

primero el elemento

de la mitad izquierda, preservando el orden relativo original de

elementos

iguales.

if izquierda[i] <= derecha[j]:

arr[k] = izquierda[i]

i += 1

else:

arr[k] = ... # TODO

j += 1

k += 1

TODO: copiar elementos restantes de izquierda

while i < len(izquierda):

arr[k] = ...

i += 1; k += 1

TODO: copiar elementos restantes de derecha

while j < len(derecha):

...

#

SECCIÓN 2: VERIFICACIÓN DE ESTABILIDAD

#

```
def merge_sort_estable_demo():
```

```
    """
```

```
    Demuestra que Merge Sort es estable:
```

```
    elementos con la misma clave mantienen su orden relativo original.
```

```
    Contexto StreamMX: registros ordenados primero por fecha, luego por puntaje.
```

```
    Si el algoritmo es estable, películas con el mismo puntaje mantienen
```

```
    el orden cronológico de la ordenación anterior.
```

```
    REFLEXIONA-2: Observamos que, al ordenar por puntaje, las películas con el mismo
```

```
    puntaje
```

```
        (por ejemplo 90 y 85) conservaron su orden relativo original (cronológico).
```

```
        Con un algoritmo inestable, esos empates podrían intercambiarse y romper
```

```
    el
```

```
    orden
```

```
        que ya se había establecido por fecha.
```

```
    """
```

```
    # Simulación: (titulo, puntaje, posicion_original_cronologica)
```

```
    registros = [
```

```
        ("Pelicula_C", 90, 1), # más antigua con puntaje 90
```

```
        ("Pelicula_A", 85, 2),
```

```
        ("Pelicula_E", 90, 3), # más reciente con puntaje 90
```

```
        ("Pelicula_B", 75, 4),
```

```
        ("Pelicula_D", 85, 5),
```

```
    ]
```

```
    # Ordenar por puntaje (clave secundaria) usando merge_sort
```

```
    # Para usar merge_sort con tuplas, necesitas una versión con comparador
```

```
    # DECISIÓN: Adaptamos merge_sort para comparar por puntaje (tupla[1]) y, en caso de
```

```
    empate,
```

```
    #         conservar el orden relativo eligiendo primero el elemento de la mitad izquierda
```

```
    #         (equivalente a usar <= en la comparación), lo cual mantiene la estabilidad.
```

```
    # TODO: implementar o adaptar merge_sort para ordenar por puntaje
```

```
    # Resultado esperado: registros con el mismo puntaje (90, 85)
```

```
    # deben mantener su orden relativo original (cronológico)
```

```
    pass
```

```
# REFLEXIONA-3: Problema de RAM abierto para la siguiente semana
```

Merge Sort es eficiente en tiempo ($O(n \log n)$), pero usa memoria extra por los arreglos auxiliares en cada merge().
El problema abierto es reducir ese consumo de RAM y el costo de copias/asignaciones,
por ejemplo con un buffer temporal reutilizable
o variantes que minimicen memoria auxiliar (tema para la siguiente semana).

#

SECCIÓN 3: BENCHMARKING vs SEMANA 8

#

```
def bubble_sort_s8(arr: list) -> None:
    """Referencia de Semana 8 para comparación empírica."""
    n = len(arr)
    for i in range(n):
        for j in range(n - i - 1):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]

def medir_tiempo(func, arr: list, *args) -> float:
    """Mide tiempo sobre una copia — el arreglo original no se modifica."""
    copia = copy.deepcopy(arr)
    inicio = time.perf_counter()
    func(copia, *args)
    fin = time.perf_counter()
    return round(fin - inicio, 6)

def generar_arreglo(n: int, distribucion: str) -> list:
    if distribucion == 'aleatorio':
        return [random.randint(0, n * 10) for _ in range(n)]
    elif distribucion == 'ordenado':
        return list(range(n))
    elif distribucion == 'inverso':
```



```
    return list(range(n, 0, -1))
    raise ValueError(f"Distribución desconocida: {distribucion}")
```

```
def benchmark_completo():
```

```
    """
    ESTRATEGIA: Esperamos una mejora enorme de Merge Sort sobre Bubble Sort
    para
    n=100 000,
    porque Bubble Sort es  $O(n^2)$  y se vuelve impráctico, mientras Merge Sort es
     $O(n \log n)$ .
    """
```

```
    La mayor diferencia debería notarse en aleatorio e inverso; en ordenado
    Bubble
    Sort
    hace menos trabajo (aunque sigue siendo cuadrático en esta versión).
```

```
    """
    # REFLEXIONA-1: Predicción vs resultados (benchmark)
    # Predijimos que Merge Sort superaría ampliamente a Bubble Sort y que Bubble
    Sort
    sería impráctico para n=100_000.
```

```
    # Los resultados lo confirmaron: Merge Sort creció de forma consistente con  $O(n \log n)$ 
```

```
(al multiplicar  $n \times 10$  el tiempo creció mucho menos que  $\times 100$ ),
```

```
    # mientras Bubble Sort se disparó hacia  $O(n^2)$  y solo fue razonable para
     $n \leq 10_000$ . La
```

```
mayor brecha se observó en inverso
```

```
    # (y también fue muy grande en aleatorio), porque Bubble Sort hace muchas
    comparaciones y, en esos casos, muchos intercambios.
```

```
tamanios    = [1_000, 10_000, 100_000]
distribuciones = ['aleatorio', 'ordenado', 'inverso']
```

```
print(f'{"Algoritmo":<18} {"n":>8} {"Distribución":<12} {"Tiempo (s)":>12}')
print("-" * 58)
```

```
for n in tamanios:
```

```
    for dist in distribuciones:
```

```
        arr = generar_arreglo(n, dist)
```

```
        t_merge = medir_tiempo(merge_sort, arr, 0, len(arr) - 1)
```

```
        # Solo incluir Bubble Sort para  $n \leq 10_000$  ( $O(n^2)$  es muy lento para 100k)
```

```
        if n <= 10_000:
```

```

t_bubble = medir_tiempo(bubble_sort_s8, arr)
print(f'{'Bubble Sort (S8)':<18} {n:>8} {dist:<12} {t_bubble:>12.6f}')

print(f'{'Merge Sort':<18} {n:>8} {dist:<12} {t_merge:>12.6f}')
print()

# VALIDA: Para  $O(n \log n)$ , al pasar de  $n=1000$  a  $n=10000$  ( $\times 10$ ),
# el tiempo debería crecer un poco más de  $\times 10$  (por el factor log), típicamente
 $\sim \times 13 - \times 15$ ,
# no cerca de  $\times 100$ . Si el crecimiento se acerca a  $\times 100$ , eso se parece más a
 $O(n^2)$ .

#
_____
_____
_____
# EJECUCIÓN
#
_____
_____
_____
if __name__ == '__main__':
    prueba = [64, 34, 25, 12, 22, 11, 90]
    copia = prueba[:]
    merge_sort(copia, 0, len(copia) - 1)
    print("Merge Sort:", copia) # Esperado: [11, 12, 22, 25, 34, 64, 90]

    print("\n--- DEMO ESTABILIDAD ---")
    merge_sort_estable_demo()

    print("\n--- BENCHMARK vs SEMANA 8 ---\n")
    benchmark_completo()

    # IA-REFLEXION-A:
    # La pista clave fue: “¿Qué esperaban que hiciera esa línea vs qué hace si ya
    sobrescribiste
    # un valor que todavía ibas a comparar?”. Ahí entendimos que al fusionar
    escribimos
    sobre arr
    # y sin auxiliares podríamos pisar datos de entrada antes de usarlos, rompiendo el
    merge.

    # IA-REFLEXION-R:

```

En el simulacro, al inicio no pudimos justificar con precisión por qué “cada nivel cuesta

$O(n)$ ” (solo dijimos “porque se fusiona todo”).

Llegamos a la respuesta al notar que en un mismo nivel los merges son sobre segmentos disjuntos y cada elemento participa una sola vez

en ese nivel; por eso la suma por nivel es $O(n)$, y con $\log_2(n)$ niveles se obtiene $O(n \log n)$.

IA-REFLEXION-C:

Descubrimos que en Quick Sort con partición Lomuto el pivote queda en posición final porque todo lo $<$ pivote queda a la izquierda y lo $\geq$ a la derecha.

Vimos que con muchos duplicados, cambiar $<$ por $\leq$ solo mueve el desbalance de lado; no evita el peor caso $O(n^2)$ en partición de dos vías.

Concluimos que para duplicados la mejora real es particionar en tres grupos (menores/iguales/mayores) o usar una selección de pivote más robusta.

Fase 2: APLICACIÓN

Reto 3 — Quick Sort + Benchmark comparativo

IA-REFLEXIÓN-A:

La IA nos aclaró el papel de i y j en Lomuto: i marca el límite del lado izquierdo y se incrementa solo cuando $A[j]$ cumple la condición.

También entendimos que el intercambio final coloca el pivote en $i+1$ y ya no se vuelve a tocar en las llamadas recursivas.

Nosotros implementamos sin ayuda el ciclo de partición (condición, $i+=1$, swaps) y las llamadas recursivas de `quick_sort` con $p-1$ y $p+1$.

Reto 4 — Quick Sort configurable: tres estrategias de pivote

IA-REFLEXION-B:

En **StreamMX** se eligió **Quick Sort con pivote mediana-de-tres** como configuración por defecto, porque ayuda a mitigar el peor rendimiento en listas casi ordenadas.

Cuando los datos llegan **frescos y sin patrón aparente**, se utiliza un **pivote aleatorio**, lo que evita depender del orden de entrada y asegura un buen desempeño esperado.

En el caso crítico de **duplicados masivos**, se planea cambiar la estrategia de partición a **tres grupos (menores, iguales y mayores)**, ya que modificar únicamente la elección del pivote no elimina el desbalance.

Fase 3: REFLEXIONA

D1 — Partición Lomuto: Traza la partición Lomuto sobre el arreglo [4, 2, 8, 3, 1]. ¿Cuántos intercambios ocurren? ¿En qué posición queda el pivote?

Ningún elemento es menor que el pivote.

Intercambios en el ciclo: 0

Intercambio final: 1

Pivote queda en índice: 0

D2 — Trampa de duplicados: ¿Por qué el arreglo [5, 5, 5, 5, 5] hace que Quick Sort con Lomuto puro sea $O(n^2)$? ¿Cuántos niveles de recursión se generan?

Nunca hay elementos menores al pivote.

Siempre se separa como $(n-1, 0)$.

Tiempo: $O(n^2)$

Recursión: n niveles

D3 — Espacio in-place: Merge Sort necesita $O(n)$ de espacio auxiliar (un arreglo extra del mismo tamaño). Quick Sort no necesita ese arreglo auxiliar ($O(1)$ de espacio extra en el arreglo). ¿Dónde se origina el $O(\log n)$ de pila de llamadas en Quick Sort y por qué no es el mismo que el $O(n)$ de Merge Sort?

Quick Sort: usa pila de recursión $\rightarrow O(\log n)$ (promedio).

Merge Sort: usa arreglo extra $\rightarrow O(n)$.

Diferencia: Quick usa llamadas, Merge usa memoria auxiliar fija.

D4 — Veredicto StreamMX: Con los datos reales del benchmark del equipo, ¿cuál algoritmo recomiendan para la distribución "ordenado"? ¿Para "duplicados"? Justifiquen con números, no con intuición.

Ordenado o casi ordenado $\rightarrow$ Merge Sort (evita $O(n^2)$)

Muchos duplicados $\rightarrow$ Merge Sort o Quick Sort 3-way

Razón: Lomuto puede degradar a $O(n^2)$.

D5 — Inestabilidad en práctica: ¿Qué ocurre con el orden de las películas de StreamMX con el mismo puntaje 7.5 después de Quick Sort? ¿Es esto un problema para el negocio? ¿Cómo lo resolverán?

Quick Sort puede cambiar el orden de iguales (no estable).

Problema si el orden entre iguales importa.

Solución: Merge Sort o desempate con índice original.

Reto IA — Reflexiona: Simulacro de defensa oral

IA-REFLEXION-R:

Lo más difícil fue hacer la traza exacta de Lomuto con el arreglo solicitado y sin confundir índices (en especial el intercambio final con $A[i+1]$).

Fase 4: VALIDA

☒ **Checklist de autoevaluación (completar antes de los casos de prueba)**

☒ **Quick Sort implementado:** `particionar_lomuto()` y `quick_sort()` completos y sin errores de sintaxis.
Si no: revisa el pseudocódigo de `COMPRENDE` y compara tu implementación línea por línea.

☒ **Benchmark ejecutado:** El equipo tiene tiempos reales para las 4 distribuciones $\times$ 3 tamaños (tabla de 12 filas).
Si no: ejecutar `ejecutar_benchmark()` y copiar los resultados en los comentarios `REFLEXIONA`.

☒ **Inestabilidad demostrada:** El experimento de estabilidad muestra una diferencia concreta entre Merge Sort y Quick Sort con `datos_streamx`.
Si no: verificar que `verificar_estabilidad()` retorna `True` con Merge Sort y `False` con Quick Sort.

☐ **Trampa de duplicados evidenciada:** El benchmark con distribución 'duplicados' muestra una caída notable de desempeño en Quick Sort (más lento o mucho más lento que Merge Sort).
Si no: verificar que la distribución 'duplicados' genera al menos 80% de valores idénticos.

☒ **Evidencias en código:** Comentarios `# COMPRENDE-1/2/3`, `# ESTRATEGIA:`, `# REFLEXIONA-1/2/3` y `# VALIDA:` completos y con contenido real.
Si no: completar los comentarios antes de presentar — los comentarios vacíos equivalen a cero en el criterio de evidencia.

☐ **Veredicto StreamMX redactado:** La recomendación cubre las tres distribuciones con datos del benchmark y menciona explícitamente tiempo, espacio, estabilidad y comportamiento con duplicados.
Si no: completar el Reto V antes de la defensa.

Reto IA — Valida: Casos adversariales para Quick Sort

IA-REFLEXION-V:

No encontramos errores: los tres casos adversariales coincidieron con nuestras predicciones y mantuvieron los invariantes (partición correcta, permutación y límites del subarreglo).